



Workloader import kit guide

From an Explorer export to unmanaged workloads and IP lists loaded into the PCE, with IP-based reconciliation and a full audit trail per run

Eric Roscher – Illumio LATAM Pre-Sales Engineering

September 2026

Working guide prepared by Eric Roscher – Illumio LATAM Pre-Sales Engineering — not an official Illumio, Inc. publication.

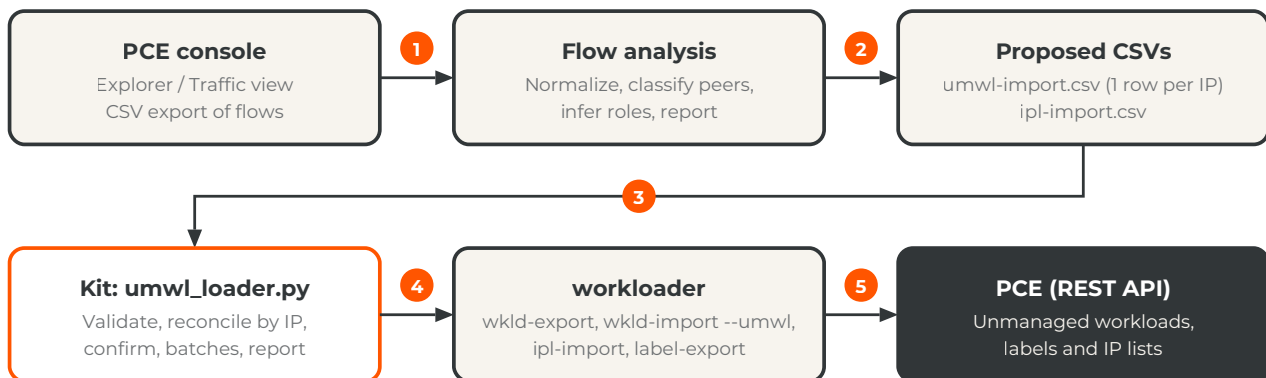
■ Contents

01	What the kit is and where it fits	3
02	Requirements and one folder per account + PCE	4
03	Initializing the working folder	6
04	Policy objects and naming	8
05	The TUI step by step	10
06	The kit and workloader	12
07	From the CSV to PCE objects	14
08	Troubleshooting and checklist	15

■ What the kit is and where it fits

The import kit closes the last stretch of a flow analysis: it takes the unmanaged workloads and IP lists proposed by the report (already in CSV format) and loads them into the PCE with `workloader`, the open-source command-line tool that Illumio maintains on GitHub¹. What the kit adds on top of `workloader` is what a bulk import does not do by itself: validate the CSV, reconcile every IP against the real PCE inventory, ask for confirmation before each write, import in batches and leave a complete record of the run.

END-TO-END FLOW: FROM THE EXPLORER EXPORT TO OBJECTS IN THE PCE



1. Export of the Explorer/Traffic view with the time window and filters of the proof of concept; open it as text, never in Excel without importing the IP columns as text.
2. Normalization (mangled IPs, dates, truncation), unmanaged workload versus IP list criterion, evidence per port, process and volume, label model and findings.
3. The report delivers the CSVs in the `wkld-import` and `ipl-import` contract, with the `R_/A_/E_/L_` naming and the evidence comment in `description`.
4. The kit calls `workloader` subcommands: read-only first (inventory), then dry run and, after confirmation, batched writes.
5. `workloader` talks to the PCE REST API using the `pce.yaml` of the working folder.

ONE WORKING FOLDER PER ILLUMIO ACCOUNT + PCE

`workloader` reads `./pce.yaml` from the folder it runs in (unless `--config-file` or `ILLUMIO_CONFIG` is set)² and the kit writes `runs/<timestamp>/` in that same folder. The isolation unit is each **Illumio account or organization + PCE** combination (each distinct `pce.yaml` profile: FQDN, port, org id, API key); sharing a folder between two of them risks importing into the wrong tenant. Structure in section 2; commands in section 3.

SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` (brian1917) — README, releases v12.1.9 — github.com/brian1917/workloader
2. `workloader` — `cmd/root.go`: configuration file lookup order (`--config-file`, `ILLUMIO_CONFIG`, `./pce.yaml`) and persistent flags — github.com/brian1917/workloader/blob/master/cmd/root.go

■ Requirements and one folder per account + PCE

The kit installs nothing on the system: it is a Python script using only the standard library, plus the workloader binary in the working folder. What it does need is an identity with write permission on the PCE and HTTPS access to its API.

REQUIREMENTS AND FOLDER STRUCTURE: ONE PER ACCOUNT + PCE

Mac (Apple Silicon or Intel)

Python 3.9+

standard library only

workloader v12.x

Intel binary + Rosetta, or go build

~/illumio/customer-a/scp57-org12/

kit/ clone of the kit repository
workloader binary; the TUI downloads it
pce.yaml ONLY this account + PCE
customer-a-umwl-import.csv
customer-a-ipl-import.csv
runs/20260903-101500/ inventory, logs, report

~/illumio/customer-a/onprem-prod/

kit/ workloader pce.yaml ... same account, another PCE

~/illumio/customer-b/<pce-or-org>/

kit/ workloader pce.yaml ... another account, same layout

1

SaaS PCE scp57

org 12 · HTTPS 443
API key with a write role

2

On-prem prod PCE

same account, other pce.yaml

3

customer-b PCE

other account, other folder

1. A SaaS PCE (one FQDN) can host several organizations; each org id is a separate tenant and gets its own folder, even when the FQDN is the same. workloader authenticates with the `pce.yaml` of the current folder and the kit shows the result of `pce-list` before going on.
2. The same customer with another PCE (on-prem, DR, another region) is another folder with its own `pce.yaml`; the kit clone can be repeated or shared through a symlink.
3. Another account: same layout, never the same `pce.yaml`.

The isolation unit: Illumio account or organization + PCE

A SaaS PCE FQDN can host several organizations (the `org id` is the number that appears in the console URL, `.../orgs/<id>/...`), and a single customer can have several PCEs: SaaS and on-prem, production and DR, regions. That is why the isolation unit is neither "the customer" nor "the PCE" but each **account or organization + PCE** combination, that is, each distinct `pce.yaml` profile (FQDN, port, org id and API key): the same customer with two orgs on the same PCE is two folders.

workloader supports several profiles in a single `pce.yaml`, with a `default_pce_name` and the `--pce <name>` flag to pick one per command⁵. The kit deliberately does not rely on that: a forgotten `--pce` or a wrong default would load the objects into the wrong tenant. Suggested naming: `~/illumio/<account>/<pce-or-org>/`, for example `~/illumio/customer-a/scp57-org12/` and `~/illumio/customer-a/onprem-prod/`.

COMPONENT	DETAIL
macOS	The macOS release (<code>mac-v12.1.9.zip</code>) is an Intel binary that runs under Rosetta ² ; for a native one, <code>brew install go</code> and <code>go build</code> from the clone. The TUI offers both paths.
Python 3.9+	Standard library only; there is no <code>pip install</code> .
workloader v12.x	Binary at <code>./workloader</code> in the working folder, on the PATH, or given with <code>--workloader <path></code> . After downloading it, remove the quarantine attribute: <code>xattr -d com.apple.quarantine workloader</code> .
Working folder	One per account or organization + PCE: it holds <code>kit/</code> (clone of the repository), <code>workloader</code> , <code>pce.yaml</code> , the CSVs and <code>runs/</code> . The scripts run from there as <code>python3 kit/umwl_loader.py ...</code> . Running inside the clone itself also works (its <code>.gitignore</code> excludes <code>pce.yaml</code> , the binary and <code>runs/</code>), but then that clone is tied to a single account + PCE.
PCE API key	User menu → My API Keys → Add ; save the <i>Authentication Username</i> (<code>api_...</code>) and the <i>Secret</i> . It inherits the user's role: creating workloads, labels and IP lists requires Global Organization Owner or Global Administrator ^{2,4} . On SaaS the <code>org_id</code> is the number in the URL.
Network	HTTPS from the Mac to the PCE FQDN (443 on SaaS). workloader needs nothing else ³ .

PCE.YAML CONTAINS THE API KEY

Do not copy it between folders or push it to a repository. The kit's `.gitignore` excludes it together with the binary, the logs and `runs/`.

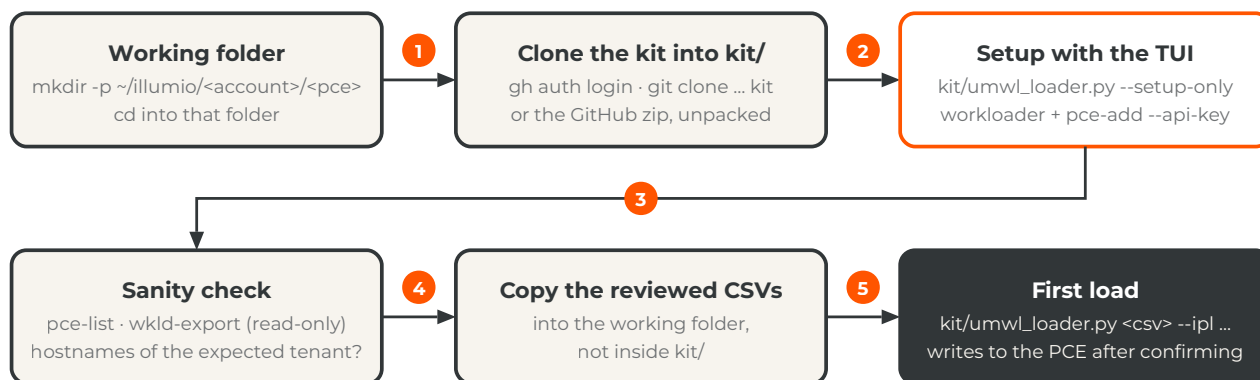
SOURCES — OFFICIAL DOCUMENTATION

- workloader v12.1.9 — release assets (mac-v12.1.9.zip, linux_amd64, linux_arm, windows) — github.com/brian1917/workloader/releases/tag/v12.1.9
- Illumio Core 24.2 — REST API — API Keys (My API Keys, the key inherits the user's role) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/REST-APIs/out/en/rest-apis/authentication-and-api-user-permissions/api-keys.html
- workloader — README: "The only requirements for workloader are the ability to run the executable and connect to the PCE over HTTPS" — github.com/brian1917/workloader
- Illumio Core 25.4 — Admin — Roles, Scopes, and Granted Access — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Admin/out/en/pce-administration/role-based-access-control/roles-scopes-and-granted-access.html
- workloader — README and `cmd/pcemgmt/addpce.go`: several PCEs in one `pce.yaml`, `default_pce_name` and the `--pce` flag — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go

■ Initializing the working folder

These steps are done once per account + PCE combination. They leave a folder with the kit clone, the workloader binary and a tested `pce.yaml`; after that, every load is a single command. The kit repository is private: a GitHub session (`gh auth login` or an SSH key loaded into the account) is needed before cloning.

INITIALIZATION: FROM AN EMPTY FOLDER TO THE FIRST LOAD



1. The folder is the isolation unit; everything that follows runs with that folder as the current directory.
2. The kit lives in `kit/`; the CSVs and `pce.yaml` sit next to it, not inside the clone. `kit/examples/` holds templates only.
3. `--setup-only` installs or verifies `./workloader`, runs `pce-add --api-key` with the values you enter and ends with `pce-list` and a connection test.
4. Before writing anything, a read-only export proves that the API key works and that the inventory belongs to the expected tenant.
5. The reviewed CSVs from the report go into the working folder, named after the customer.
6. The TUI walks through steps 0 to 10 of section 5; it writes to the PCE only after the dry-run confirmation.

Commands

TERMINAL · A. WORKING FOLDER FOR THIS ACCOUNT + PCE

```
mkdir -p ~/illumio/customer-a/scp57-org12 && cd ~/illumio/customer-a/scp57-org12
```

TERMINAL · B. THE KIT INSIDE THE FOLDER (PRIVATE REPOSITORY)

```
gh auth login # once; or an SSH key loaded into GitHub
git clone https://github.com/roschereric/illumio-workloader-import-kit.git kit
# equivalent: gh repo clone roschereric/illumio-workloader-import-kit kit
# without git: on GitHub, Code → Download ZIP, and unpack it as kit/
```

Expected result. The scripts run from the working folder as `python3 kit/umwl_loader.py ...`: the current directory is the working folder, so `./workloader`, `./pce.yaml` and `./runs` sit next to the CSVs and not inside the clone¹. The CSVs live in the working folder, not in `kit/`; `kit/examples/` contains templates only. Running inside the clone also works (the sensitive files are in `.gitignore`), at the cost of tying the clone to one account + PCE.

RESULTING LAYOUT

```
~/illumio/customer-a/scp57-org12/
kit/                # repository clone; update with: git -C kit pull
workloader          # binary; the TUI downloads it (or a symlink to a shared one)
pce.yaml            # ONLY this account + PCE; created by pce-add; never copy it
customer-a-umwl-import.csv
customer-a-ipl-import.csv
runs/               # one subfolder per run
```

TERMINAL · C. WORKLOADER AND PCE.YAML WITH THE TUI

```
python3 kit/umwl_loader.py --setup-only
```

The TUI looks for `./workloader` (then the PATH, or the `--workloader` path). If it does not find it, it offers to download the release (`mac-<version>.zip`, with `curl + unzip` and `xattr -d com.apple.quarantine`) or to clone the workloader repository into `workloader-src/` and build it with `go build`. Then it runs `pce-list`; if there is no `pce.yaml`, it asks for a short name, FQDN, port, API user, API secret and org id, runs `pce-add --api-key ...` with those values², shows `pce-list` again and tests the connection with a `label-dimension-export`.

TERMINAL · D. SANITY CHECK (READ-ONLY)

```
./workloader pce-list
./workloader wkld-export --output-file sanity.csv
# look at the hostnames in sanity.csv: is this the inventory of the expected tenant?
```

TERMINAL · E. REVIEWED CSVS AND FIRST LOAD

```
cp ~/Downloads/customer-a-umwl-import.csv ~/Downloads/customer-a-ipl-import.csv .
python3 kit/umwl_loader.py customer-a-umwl-import.csv --ipl customer-a-ipl-import.csv --priority 1
```

SITUATION	WHAT TO DO
f. Another account or another PCE	Repeat a–d in a new folder. Never copy <code>pce.yaml</code> . If you prefer a single kit clone, share it through a symlink: <code>ln -s ~/illumio/kit-shared kit</code> .
g. Updating the kit	<code>git -C kit pull</code> (or download the zip again). The loader has no other dependencies: there is nothing else to install.
Several PCEs in the same <code>pce.yaml</code>	Not the kit's scheme. If it exists anyway, always pass <code>--pce <name></code> to <code>umwl_loader.py</code> ; the TUI shows <code>pce-list</code> and asks whether it is the right PCE before going on.

SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` — `cmd/root.go`: the configuration file is `--config-file`, else `ILLUMIO_CONFIG`, else `./pce.yaml` in the current directory — github.com/brian1917/workloader/blob/master/cmd/root.go
2. `workloader` — `cmd/pcemgmt/addpce.go`: `pce-add` with `--api-key` prompts for (or takes by flag) `--api-user`, `--api-secret` and `--org`; without it, it asks for email and password — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
3. GitHub CLI — `gh auth login` and `gh repo clone` — docs.github.com/en/github-cli/github-cli/about-github-cli

■ Policy objects and naming

The kit creates two kinds of object and, as a side effect, label values. Knowing what each one is explains the decisions of the analysis and those of the TUI.

OBJECT	WHAT IT IS AND WHEN IT IS USED
Unmanaged workload (UMWL)	A network entity without a VEN that is registered in the PCE so that rules can be written about it; policy is enforced by the rules on the side that has a VEN ¹ . For specific servers with an identifiable role (DNS, NTP, Zabbix, Splunk, NetBackup, databases, load balancers, bastions, scanners). One per IP, with labels.
IP list	A static collection of addresses, ranges and FQDNs ² . For broad peers or peers that are not servers : user VLANs, whole subnets, cloud metadata (169.254.169.254), public NTP, SaaS consoles. Also as a shortcut to write a rule today that migrates to labels later.
Labels and label types	Four default types (Role, Application, Environment, Location) plus any created in the PCE ³ . workloader creates new values, not types : a column that is not an existing type is silently ignored (the TUI detects it).
Services	Port and protocol objects. The report proposes them for the rules; the kit does not create them (they are loaded with <code>svc-import</code> or from the console).

WHY THE PRIORITY GOES IN THE DESCRIPTION

It is the only free-text field that travels with the object to the PCE and can be read afterwards in the console. The kit uses it to filter (`--priority 1`) without needing extra columns that `workloader` would ignore.

Naming convention

FIELD	RULE	EXAMPLE
Labels	Prefix by type: R_ role, A_ application, E_ environment, L_ location	R_DNS · A_CoreInfra · E_Prod · L_OCI
name	Descriptive role followed by the IP; it is what the console shows and what makes the row unique	Zabbix Server 10.43.43.21
hostname	Empty. The FQDNs in the exports may be anonymized; an invented hostname is misleading	(blank)
interfaces	eth0:<ip> ; several interfaces separated by ;	eth0:10.43.43.21
description	Prefix [<group> P<priority> conf:<Alta Media Baja>] followed by the evidence comment	[C3 P1 conf:Alta] Zabbix server: polls 10050/TCP...

SOURCES — OFFICIAL DOCUMENTATION

1. Illumio Core 24.5 — Security Policy — Workload Setup Using PCE Web Console (Add Unmanaged Workload) — product-docs-repo.illumio.com/Tech-Docs/Core/24.5/Security-Policy/out/en/security-policy-guide-24-5/workloads/workload-setup-using-pce-web-console.html
2. Illumio Core 24.2 — Getting Started — Policy Objects — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Getting-Started/out/en/policy-overview/policy-objects.html
3. Illumio Core 25.4 — Security Policy — Create a Label Type — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Security-Policy/out/en/security-policy-objects/about-labels-and-label-groups/label-types/create-a-label-type.html

■ The TUI step by step

Two commands cover normal use, always from the working folder (section 3). The first runs once per folder; the second, once per CSV.

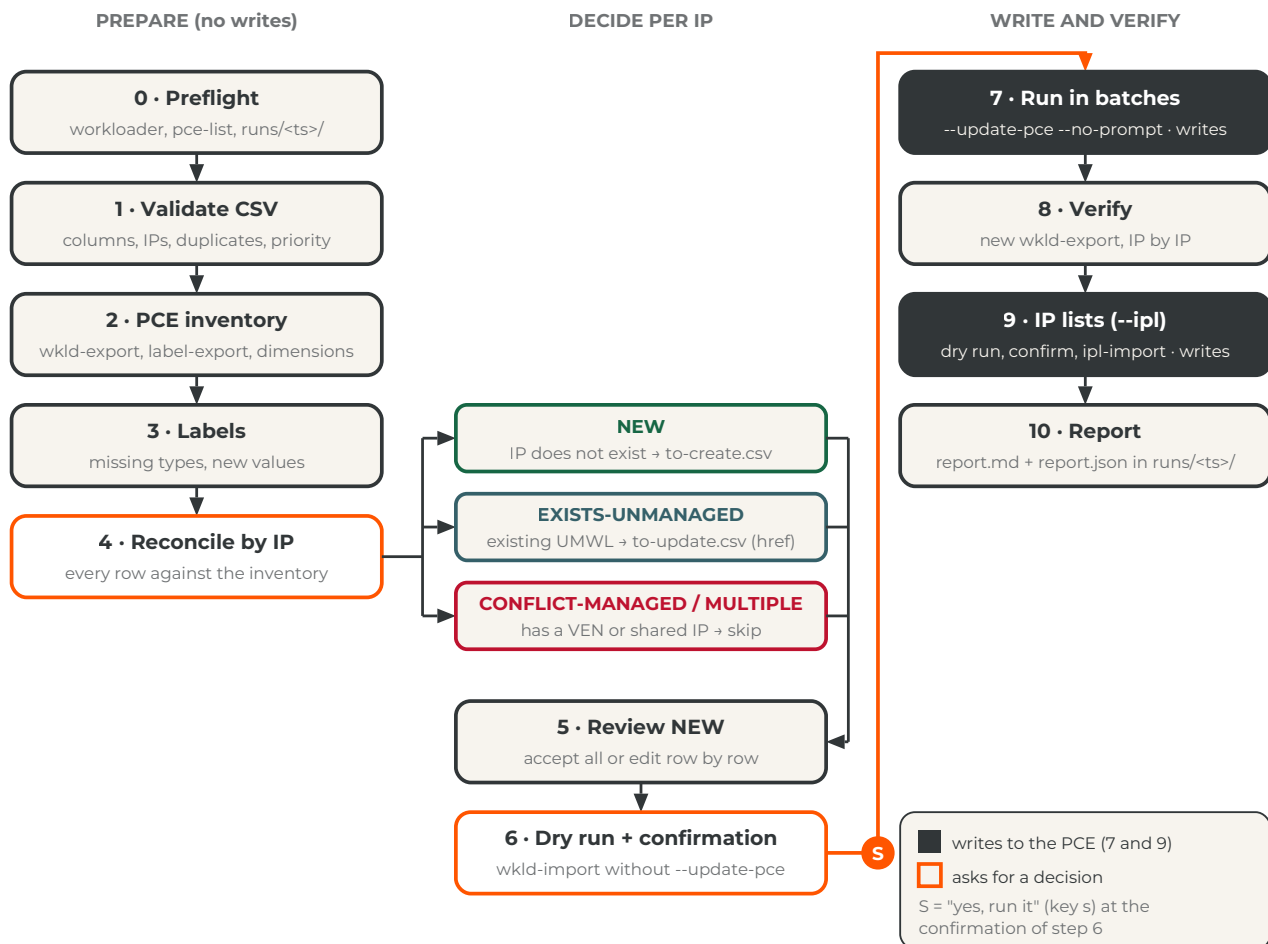
TERMINAL · WORKING FOLDER OF THE ACCOUNT + PCE

```
# 1. install/verify workloader and configure pce.yaml (once per account + PCE)
python3 kit/umwl_loader.py --setup-only

# 2. load the priority-1 workloads and, at the end, the IP lists
python3 kit/umwl_loader.py customer-a-umwl-import.csv --ipl customer-a-ipl-import.csv --priority 1

# other flags: --pce NAME --chunk 20 --runs ./runs
#              --workloader PATH (default ./workloader, then the PATH)
```

TUI SEQUENCE AND DECISION POINTS



What each step decides

STEP	DECISION OR RESULT
3 · Labels	A column that is not a PCE label type: discard it, or exit and create the type with <code>label-dimension-import</code> . New values: listed and confirmed before going on.
4 · Reconcile	For each EXISTS or CONFLICT: update the labels and description of the existing one (by <code>href</code>), skip , create anyway (deliberate duplicate), rename and update , or apply the same decision to all of the same kind. Default: update the EXISTS-UNMANAGED, skip the CONFLICT.
5 · Review NEW	Accept all or review one by one: keep, edit (name, hostname, description, labels), skip, accept the rest.
6 · Dry run	The workloader log lines with what it would create or change are shown. The "Run against the PCE?" question defaults to no : you have to type "s" (yes) to write.
7 · Batches	Each batch of <code>--chunk</code> rows is one <code>wkld-import</code> call. If a batch fails: retry, skip it (it is recorded) or abort. The batch CSV stays in <code>runs/<ts>/create-chunkNNN.csv</code> .

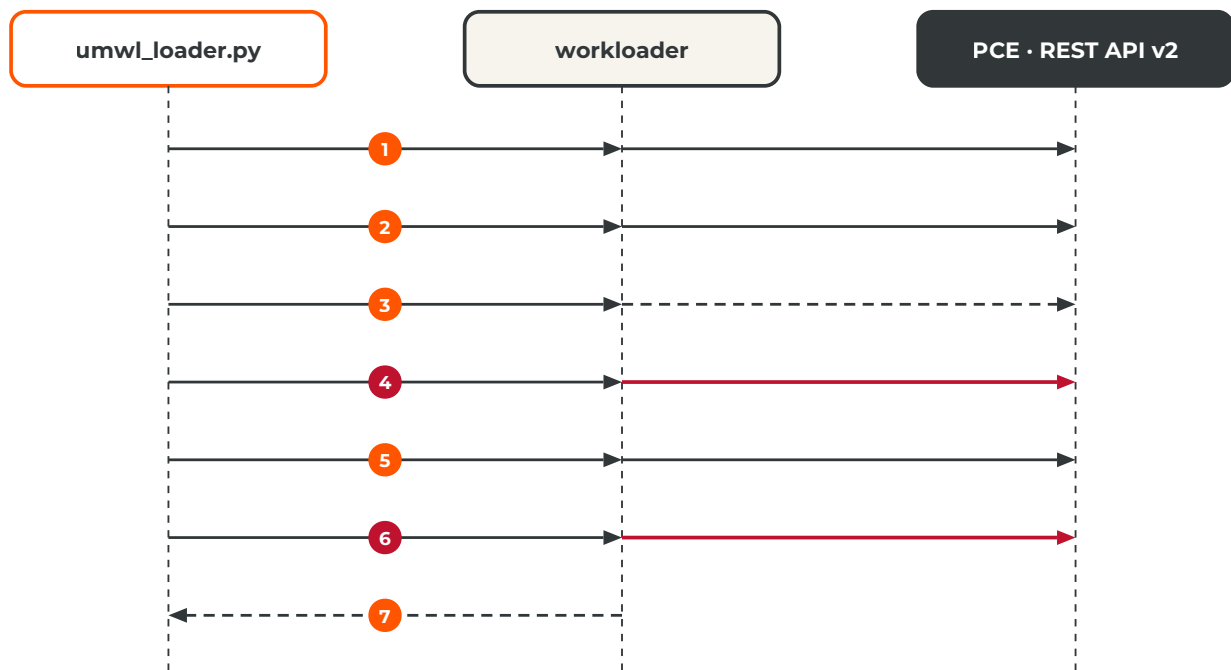
SOURCES — OFFICIAL DOCUMENTATION

1. [workloader — README: --update-pce and --no-prompt](#) ("auto-accept this prompt, as would be needed in automation"); logs in `workloader.log` — github.com/brian1917/workloader
2. [workloader — cmd/root.go: persistent flags --pce, --log-file, --update-pce, --no-prompt, --config-file](#) — github.com/brian1917/workloader/blob/master/cmd/root.go

■ The kit and workloader

The kit does not talk to the PCE API: it orchestrates workloader subcommands (the `./workloader` binary in the working folder, or the one on the PATH, or the one given with `--workloader`) and reads their logs. Every call runs with `--log-file runs/<ts>/<step>.log` and, if specified, with `--pce <name>`. Without `--update-pce`, no workloader command modifies the PCE¹; the kit adds that flag (together with `--no-prompt`) only in steps 7 and 9, after explicit confirmation.

WHO CALLS WHOM: KIT, WORKLOADER AND PCE



Red line = write to the PCE (only after confirmation). Dashed line = read-only / log.

1. `pce-list` (and `pce-add --api-key ...` if there is no `pce.yaml`); workloader authenticates with the API key and the kit tests the connection with a `label-dimension-export`.
2. `wkld-export`, `label-export`, `label-dimension-export` into `runs/<ts>/`: workload inventory (with `managed`, `ven_href`, `interfaces`, `public_ip`), labels and types. Read-only.
3. `wkld-import to-create.csv --umwl --update=false` and `wkld-import to-update.csv` without `--update-pce`: workloader reads the PCE to compute the plan and writes it to the log; it modifies nothing.
4. For each batch: `wkld-import create-chunkNNN.csv --umwl --update=false --update-pce --no-prompt` (and the same for the update batches, which match by `href`). Creates unmanaged workloads and any missing label values.
5. `wkld-export` again: the kit checks that every created IP appears in the inventory.
6. With `--ipl`: `ipl-import file.csv` (dry run) and, after confirmation, `ipl-import file.csv --update-pce --no-prompt`.
7. Every call leaves its log in `runs/<ts>/`; the kit filters the "to be created", "bulk create workload successful", "[WARN]" and "[ERROR]" lines and shows them on screen.

The same steps without the TUI

TERMINAL · MANUAL EQUIVALENTS (SAME WORKING FOLDER, SAME PCE.YAML)

```
# PCE (once per account + PCE) and inventory
./workloader pce-add --api-key --name customer-a --fqdn pce.customer-a.example --port 443 \
  --api-user api_xxx --api-secret '...' --org 1
./workloader wkld-export --output-file pce-workloads.csv
python3 kit/reconcile_umwl.py pce-workloads.csv customer-a-umwl-import.csv

# dry run (read workloader.log), then write
./workloader wkld-import customer-a-umwl-import-to-create.csv --umwl
./workloader wkld-import customer-a-umwl-import-to-create.csv --umwl --update-pce

# existing ones: labels and description only, matched by href
./workloader wkld-import customer-a-umwl-import-existing.csv --update-pce
./workloader ipl-import customer-a-ipl-import.csv --update-pce
```

PCE-ADD NEEDS --API-KEY TO USE AN API KEY

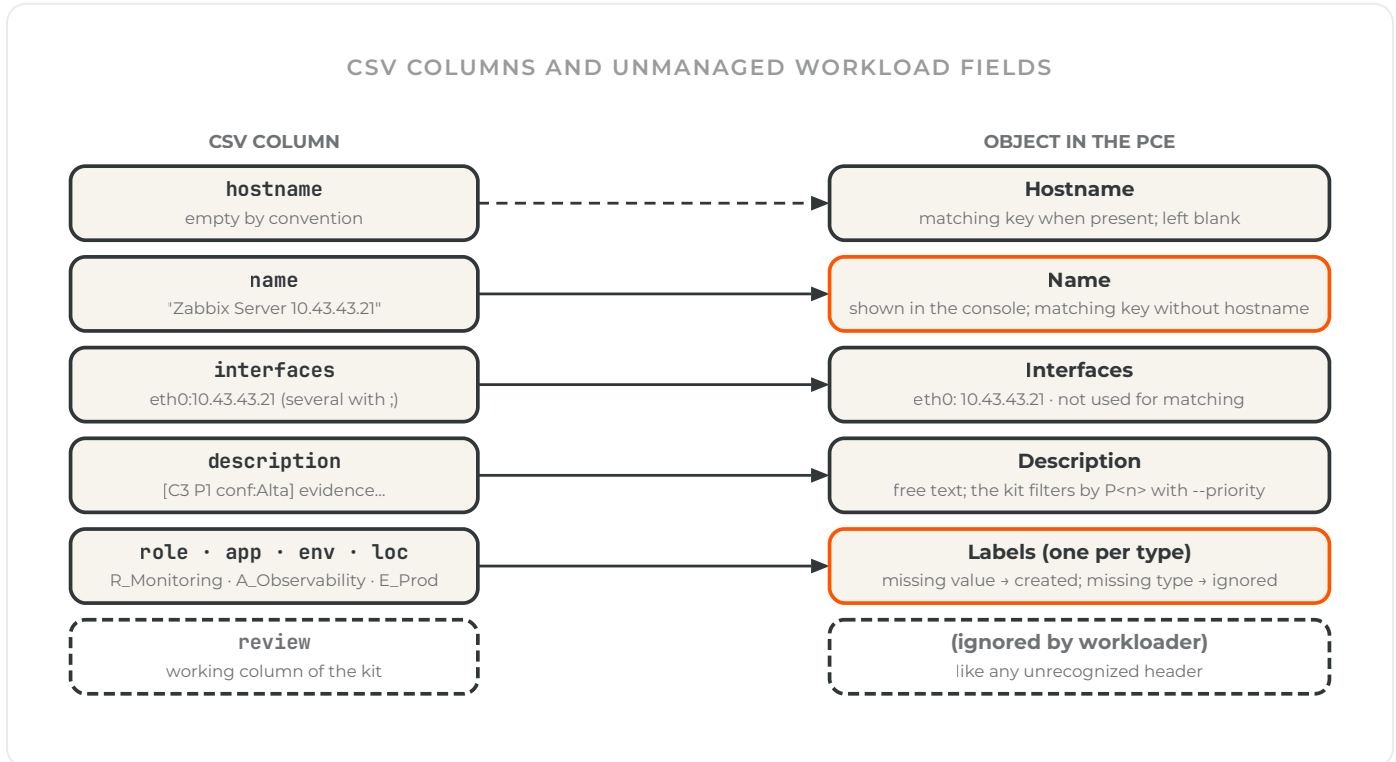
In the workloader code, `--api-user`, `--api-secret` and `--org` are only read when the `--api-key` flag is present; without it, the command asks for email and password². The kit version published in the repository already includes it.

SOURCES — OFFICIAL DOCUMENTATION

1. workloader — README: "When a command modifies resources in the PCE, workloader does not trigger the action unless the `--update-pce` flag is included" — github.com/brian1917/workloader
2. workloader — cmd/pcemgmt/addpce.go: pce-add flags (`--api-key`, `--api-user`, `--api-secret`, `--org`, `--disable-tls-verification`) — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
3. workloader — cmd/wkldexport/headers.go: export columns (hostname, name, interfaces, public_ip, href, managed, ven_href...) — github.com/brian1917/workloader/blob/master/cmd/wkldexport/headers.go

■ From the CSV to PCE objects

`wkld-import` recognizes a fixed set of headers (`href` , `hostname` , `name` , `interfaces` , `public_ip` , `description` , `enforcement` , `visibility` and others) plus **one column per existing label type**; everything else is ignored¹. The diagram shows which workload field each column goes to.



Matching: by href, hostname or name, never by IP

Before creating, workloader checks whether the row already exists: by `href` if the column is present, else by `hostname`, else by `name` (`--match` forces one of them)¹. The IP plays no part. That is why two rows with the same `name` and no hostname are the same workload for workloader (the kit appends the IP as a suffix), and why a UMWL that already exists under another name would be created as a duplicate unless it is first reconciled by IP against the `wkld-export`: that is step 4 of the TUI and the only job of `reconcile_umwl.py`.

IP lists. The `ipl-import` contract is `name,description,include,exclude,fqdns`: `name` is the matching key when there is no `href` (if it exists, it updates; if not, it creates)² and by convention it is lowercase with the `ipl-` prefix; `include` and `exclude` carry IPs, CIDRs or ranges separated by `;`; `fqdns`, names where applicable; `description`, the reasoning from the report.

SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` — `cmd/wkldimport/cmd.go`: recognized headers, interfaces format, `--umwl`, `--update`, `--match` (`href`, `hostname`, `name`, `external_data`) — github.com/brian1917/workloader/blob/master/cmd/wkldimport/cmd.go
2. `workloader` — `cmd/iplimport/iplimport.go`: headers `name`, `description`, `include`, `exclude`, `fqdns`; `;"` separator; matching by `href` and `name` — github.com/brian1917/workloader/blob/master/cmd/iplimport/iplimport.go

■ Troubleshooting and checklist

SYMPTOM	CAUSE AND FIX
macOS refuses to open <code>workloader</code> ("the developer cannot be verified")	Gatekeeper quarantine on the downloaded zip. <code>xattr -d com.apple.quarantine ./workloader && chmod +x ./workloader</code> . The TUI does this when it downloads.
Apple Silicon	The release is an Intel binary and runs under Rosetta 2. For a native binary: <code>brew install go</code> , clone the repository and <code>go build -o ../workloader</code> ; the TUI offers this option.
<code>pce-add</code> asks for email and password	<code>workloader pce-add</code> was run by hand without <code>--api-key</code> ; without that flag <code>workloader</code> attempts a username/password login against login.illum.io (401 for SSO accounts) and ignores <code>--api-user</code> , <code>--api-secret</code> and <code>--org</code> . The TUI always adds <code>--api-key</code> on both of its options.
The dry run works and the import returns 401/403	The API key inherits the user's role. A global role with write access is needed (Global Organization Owner or Global Administrator) or an equivalent service account. Check the <code>org id</code> too.
<code>pce-list</code> shows another customer or another organization	Wrong folder (the current directory decides which <code>pce.yaml</code> is used) or <code>ILLUMIO_CONFIG</code> pointing to another file. One folder per account + PCE; <code>unset ILLUMIO_CONFIG ; --pce <name></code> only if the <code>pce.yaml</code> has several profiles.
Only one of several workloads with the same name was loaded	Same <code>hostname</code> (or same <code>name</code> without hostname) = same workload for <code>workloader</code> . Make the <code>name</code> unique; the TUI appends the IP as a suffix.
A label column was not applied	The type does not exist in the PCE. Create it in the console or with <code>./workloader label-dimension-import types.csv --update-pce</code> (CSV with <code>key</code> and <code>display_name</code>) ¹ and run again.
The Explorer export has exactly N round rows	It is truncated at the query limit. Raise the limit, filter by application, exclude scanners, shorten the window and export again; the console shows up to 100,000 results and the CSV up to 200,000 on a standalone PCE ² .
TLS error with an on-prem PCE	Internal certificate. Install the CA in the keychain; <code>--disable-tls-verification true</code> only in a lab.

Before writing to the PCE

- ☐ **Right folder** — the current directory is the account + PCE folder, `pce-list` shows that PCE (FQDN and org) and `ILLUMIO_CONFIG` is not set.
- ☐ **CSV in the v2 format** — empty hostname, unique `name` with IP, `interfaces` as `eth0:<ip>`, prefixed labels, priority in the description.
- ☐ **Label types** — every label column in the CSV exists in the PCE (step 3 without warnings).
- ☐ **Reconciliation reviewed** — no CONFLICT-MANAGED is updated without having looked at it; the EXISTS-UNMANAGED keep the PCE name unless explicitly decided otherwise.
- ☐ **Dry run read** — the "to be created" lines match what is expected and there is no "[ERROR]".
- ☐ **Small batch the first time** — `--chunk 5` on the first run against a new PCE to limit the impact of an error.
- ☐ **Report archived** — `runs/<ts>/report.md` stays in the customer folder; it is not pushed to the repository.

SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` — `cmd/labeldimension/import.go`: `label-dimension-import` (headers key, display_name, href...) — github.com/brian1917/workloader/blob/master/cmd/labeldimension/import.go
2. Illumio Core 25.4 — Visualization — About the Visualization Tools (Explore, Traffic, result limits, asynchronous queries) — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Visualization/out/en/visualization-tools/about-the-visualization-tools.html
3. Illumio Core 22.5 — REST API — Explorer: `traffic_flows/async_queries`, `max_results 200,000` — product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html

Code, sample CSVs and this guide



Repository `illumio-workloader-import-kit`: README with the Explorer step by step and the prompts to replicate the analysis.

ABOUT THIS DOCUMENT

This guide was prepared by Eric Roscher – Illumio LATAM Pre-Sales Engineering to explain how to use the unmanaged workload and IP list import kit with `workloader`. It is a working aid for the recipient and is not an official publication of Illumio, Inc.; it does not modify or replace official product documentation, contractual agreements, or statements of work. Validate all technical details against the official documentation for your specific product versions before acting on them. Illumio and the Illumio logo are trademarks of Illumio, Inc., used here to identify the subject matter.